



Execution-Grounded LLM Coding Agents: From Code Generation to Autonomous Software Engineering

Tong Zhu 225040538

School of Data Science

Chinese University of Hong Kong, Shenzhen

CSC6032 Advanced Operating System, Spring 2026





Topic: Execution-Grounded LLM Coding Agents

- **Paper 1** (Foundation) —Codex: Evaluating LLMs Trained on Code *arXiv 2021*
- **Paper 2** (Systems Bridge) —CodeAct: Executable Code Actions Elicit Better LLM Agents *ICML 2024*
- **Paper 3** (Recent) —OpenHands: An Open Platform for AI Software Developers as Generalist Agents *ICLR 2025*

Narrative arc: Code generation capability → code execution as the right action modality for agents → an open platform for generalist agents operating in a sandboxed runtime





Background: Why LLMs for Code?

- Software engineering is highly structured — programs have formal semantics and deterministic execution.
- Pre-2021: neural code synthesis limited to small snippets.
- **Key insight:** large-scale pre-training on public code (GitHub) transfers to unseen tasks.
- Unlocks functional code generation, test writing, and eventually autonomous debugging.

Table 1. Evolution of neural code generation approaches.

Era	Approach	Capability
<2020	Rule-based	Templates
2020–21	GPT-2/3 fine-tune	Snippets
2021+	Codex-scale	Functions
2023+	Agent loops	Full tasks





“Evaluating Large Language Models Trained on Code”

Chen et al., OpenAI. arXiv:2107.03374 (2021)

Core contribution:

- GPT-based model (~12 B params) fine-tuned on **159 GB** of filtered public GitHub Python code.
- Introduces **HumanEval** —164 Python programming problems with unit tests; measures *pass@k*.
- Establishes execution-based code evaluation as a standard way to measure code generation.

Why it matters: LLM + massive code corpus → general-purpose code generation.
HumanEval becomes a widely used benchmark for code generation.





Codex — HumanEval Benchmark Design

- 164 hand-written Python problems, never published online.
- Each problem: docstring + function signature → correct implementation.
- Evaluated by *executing* the model's output against unit tests.
- Metric: **pass@k** — does at least one of k samples pass all tests?
- Avoids memorization bias from train-set contamination.

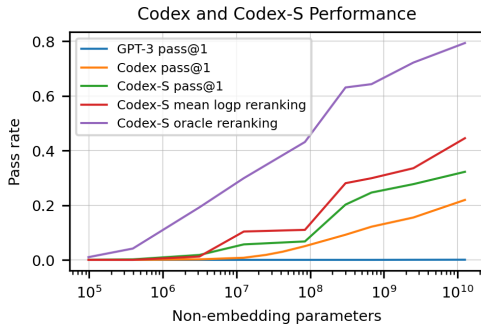


Figure 1. pass@k results on HumanEval (Chen et al., 2021).





Codex —Key Findings & Limitations

Findings:

- Scale improves code generation capability.
- Clear docstrings strongly affect correctness.
- Execution-based evaluation exposes reliability gaps.

Limitations:

- Single function —no multi-step reasoning.
- Model cannot execute code or self-correct.

⇒ Can we let the LLM *execute* code as part of its reasoning?

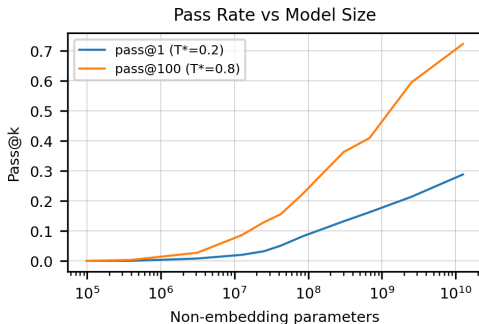


Figure 2. pass@k scales with model size (Chen et al., 2021).





“Executable Code Actions Elicit Better LLM Agents”

Wang et al. ICML 2024

Core contribution:

- Proposes **CodeAct**: let the agent emit executable *Python code* as its action.
- Replaces JSON/text action formats with executable code inside a **Python interpreter**.
- **Agent observes stdout/stderr as feedback and iterates** —closing the execution loop.
- Also releases **CodeActAgent**, with 7B models fine-tuned from LLaMA-2 and Mistral.

Systems connection: Actions become executable programs that interact with tools, packages, and external environments through code.





CodeAct — Action Space Comparison

Table 2. Comparison of action formats for LLM agents.

Property	Natural Language	JSON Tool Calls	CodeAct
Executability	Needs parser	Needs dispatcher	Directly executable
Composability	Weak	One tool per call	Loops, conditionals, helpers
State	Implicit in prompt	Usually external only	Variables persist in runtime
Error feedback	Ambiguous text	Tool-specific strings	Python traceback
Turn efficiency	Multi-turn planning	Many small calls	One script can do more

Same task: count rows in all CSV files

JSON agent: list files → read each file → count rows

Why CodeAct wins

- One action can express the whole procedure.
- Runtime state stays available for the next step.
- Failures become actionable execution feedback.

Illustrative example adapted from Wang et al., ICML 2024, Figure 2.





CodeAct — A Concrete Interaction Example

Task: “Find all CSV files in the project and count total rows.”

Agent emits:

```
import glob
files = glob.glob(
    "**/*.csv", recursive=True)
total = 0
for f in files:
    with open(f) as fh:
        total += sum(1 for r in fh) - 1
print(files, total)
```

Observation (stdout):

```
['data/a.csv',
'logs/b.csv'] 1843
```

Agent's final reply:

“Found 2 CSV files, 1843 data rows total.”

- One action replaces 3–4 sequential JSON tool calls.
- State persists across turns; tracebacks enable self-correction.





CodeAct Results and Transition to OpenHands

- **M³ToolEval**: 82 human-curated multi-turn tool-use tasks.
- For gpt-4-1106-preview, CodeAct reaches **74.4%** vs. 52.4% for JSON and 53.7% for text.
- Fine-tuned 7B CodeActAgent models improve open-source agent performance, especially with the Mistral backbone.
- Fewer turns: code compresses multiple actions into one.
- **Next question**: what runtime and agent framework can support this interaction model?
- **OpenHands** provides a sandboxed runtime, an event stream, and a hub of reusable agents.





CodeAct — Zero-Shot Performance

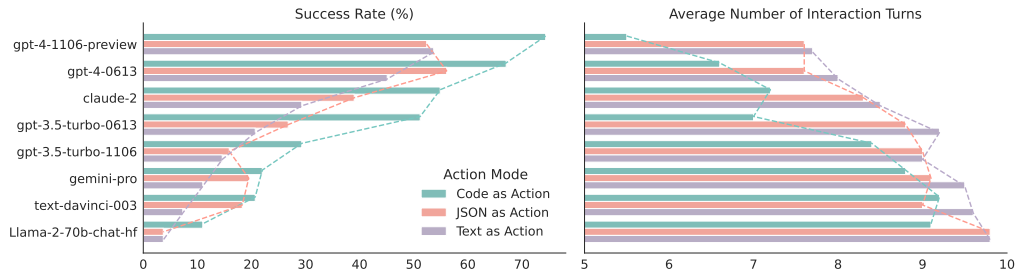


Figure 4. Zero-shot model performance across action formats (Wang et al., ICML 2024).





Paper 3 —OpenHands (Wang et al., ICLR 2025)

- Open-source platform for building and evaluating generalist and specialist agents.
- **Docker sandbox**: each session gets an isolated container; no shared state.
- **EventStream**: tracks the history of actions, observations, and user interactions.
- **Runtime**: bash shell, IPython server, and Chromium browser inside the sandbox.
- **AgentHub + AgentSkills**: reusable agents and tools share the same runtime abstraction.



OpenHands — Architecture

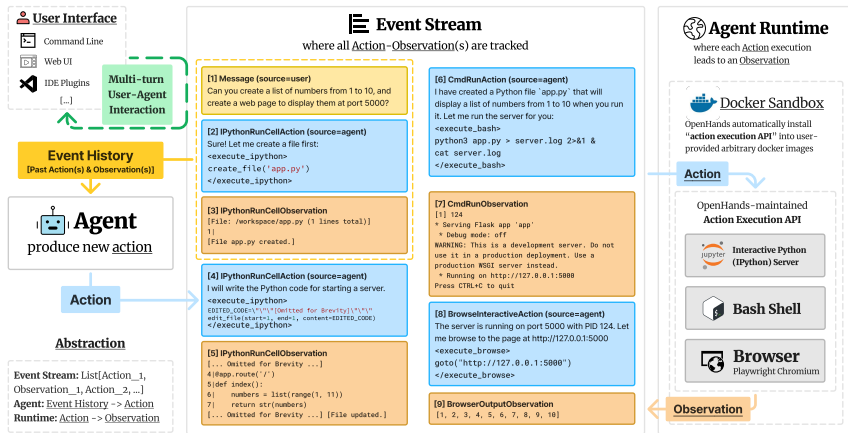


Figure 5. OpenHands architecture (Wang et al., ICLR 2025).



CodeActAgent:

- Default generalist agent in OpenHands.
- Can converse with the user or act through code.
- Uses bash, Python, and browser actions in one framework.

BrowsingAgent:

- Generalist web agent used as a baseline for web tasks.
- Similar in spirit to WebArena-style agents.

OpenHands also supports multi-agent delegation through a dedicated `AgentDelegateAction`.

Table 2. Agent types described in the OpenHands paper.

Agent	Role
CodeActAgent	Default generalist
BrowsingAgent	Web-task baseline
GPTSwarm Agent	Graph-based multi-agent
Micro Agent	Specialized prompt agent





- **SWE-Bench Lite**: 300 real GitHub issues; task is to produce a passing patch.
- OpenHands + Claude-3.5 Sonnet: **26.0%** success rate.
- Comparable to Aider (26.3%) and above SWE-Agent (18.0%) in this table.
- Paper emphasizes breadth across benchmarks, not a single dominant SWE score.

Table 3. SWE-Bench Lite success rates from the OpenHands paper (ICLR 2025).

System	Success rate
SWE-Agent (GPT-4)	18.0%
Aider (GPT-4o + Claude-3 Opus)	26.3%
OpenHands (Claude-3.5)	26.0%





Table 4. Summary of three papers and their OS connections.

	Paper	Key idea	OS connection
1	Codex	LLMs generate correct code	Execution is the evaluator
2	CodeAct	Code is the agent action space	Executable code in a live interpreter
3	OpenHands	Open platform for generalist agents	Docker sandbox, event stream, runtime

- **Codex** establishes that LLMs *can* generate correct code at scale.
- **CodeAct** establishes that LLMs *should* act through code —not JSON or NL.
- **OpenHands** turns the paradigm into a reusable platform: sandboxed runtime, event stream, and shared agent abstractions.





Open Problems & Future Directions

- **Long-horizon reliability:** agents still fail on tasks requiring 50+ steps; error compounds.
- **Sandboxing vs. capability:** tighter isolation limits what agents can do (network, GUI, external APIs).
- **Evaluation coverage:** SWE-Bench is Python-centric; multilingual, multi-repo tasks remain hard.
- **Cost and latency:** cloud LLM calls at every step are expensive; smaller local models lag behind.
- **Safety:** autonomous agents with bash access can delete files, exfiltrate data; need OS-level permission models.
- **Human-agent collaboration:** when should the agent ask vs. proceed? Optimal interruption policies are open.





- LLM coding agents have evolved rapidly: from single-function generation (Codex, 2021) to OS-aware autonomous engineering (OpenHands, 2025).
- The key insight of CodeAct —*use executable code as the action language*—turns the LLM into a genuine computer-using agent.
- OpenHands shows how a sandboxed runtime and event stream can support reusable agent execution across tasks.
- Remaining challenges sit at the intersection of systems, safety, and long-horizon planning.

Thank you

